ChatGPT

Centralized Marketing Tracking Integration (Meta, TikTok, Google) – CodeIgniter 4

Executive Summary: We will build a **single, first-party tracking layer** in CodeIgniter 4 that captures all key e-commerce events in one place (frontend & backend) and forwards them to Meta (Pixel + Conversions API), TikTok (Pixel + Events API), and Google (GTM→GA4/Google Ads). The developer will add a **Marketing Settings** panel (for IDs, tokens, toggle, test mode, etc.), instrument the site's e-commerce workflow to fire events (ViewContent, AddToCart, InitiateCheckout, Purchase, etc.), and implement a **MarketingService** that dispatches each event both via browser tags and server APIs with proper deduplication (using `event_id`). We will also log events in the DB for debugging. This guide covers **required tokens/IDs per platform**, **standard event names and parameters**, example payloads, deduplication strategy, user-data hashing, CI4 architecture (routes, controllers, services, models, DB), admin UI fields, dataLayer/JS snippets, server-endpoint code, COD handling, test/debug UI, security/rate-limit considerations, and a full developer action checklist.

1. Platform Credentials & APIs

Each ad platform requires specific IDs and tokens (store securely; never expose in JS). The admin panel should collect:

Platform	Required IDs/Tokens	Notes
Meta	Meta Pixel ID (`pixel_id`) Meta Conversions API Access Token	Obtain from Facebook Business Manager. CAPI token is from Events Manager – use server-to-server API.
TikTok	TikTok Pixel ID (`pixel_code`) TikTok Events API Access Token	Obtain from TikTok Ads → Events Manager. Use their "Events API" endpoint.
Google	Google Tag Manager Container ID (`GTM-XXXX`) GA4 Measurement ID (e.g. `G-XXXX`) Google Ads Conversion ID & Label (optional)	Use GTM for all analytics (GA4 and Google Ads). If using Google Ads conversion tracking (optional), store its Conversion ID/Label from Google Ads.

All credentials (Pixel IDs, API tokens) should be stored **encrypted** in the database (or in `.env` with database reference). The Marketing panel should allow the admin to *toggle tracking on/off* and enter these values. Example admin fields: *Meta Pixel ID, Meta CAPI Token, TikTok Pixel ID, TikTok Events Token, GTM ID, GA4 ID, Google Ads ID/Label, Test Mode On/Off*.

2. Events to Track (Browser + Server)

We will instrument the following **standard e-commerce events**. Each is sent *once per action*, both via the browser (Pixel/GTM) and via server (Conversions API/Events API), using the same `event_id` for deduplication:

- **PageView** – page load (all pages).
- **ViewContent** (Meta/TikTok) / **view_item** (GA4) – customer views a product detail page.
- **Search** – user searches product (TikTok & GA4).
- **AddToCart** – adds a product to cart.
- **ViewCart** (GA4 only) – viewing cart.
- **InitiateCheckout** (Meta/TikTok) / **begin_checkout** (GA4) – starts checkout.
- **AddPaymentInfo** – adds payment method (Meta) / **add_payment_info** (GA4).
- **Purchase** – completes an order. *(For COD, see below.)*
- **CompleteRegistration** (Meta) – account signup.
- **Lead/Contact/SubmitForm** – (generic lead events, e.g. contact form).
- **Custom events** – coupon applied, item removed from cart, order cancelled/refunded.

For each event, send relevant parameters (see tables below). Always include `currency` and `value` for commerce events【1†L92-L99】【38†L125-L133】 (for GA4, currency is *required* when sending a revenue `value` 【38†L125-L133】). Use consistent parameter names/values between the Pixel (browser) and API (server) for each event【1†L82-L89】.

2.1. Event Parameters

Important parameters (fields) for e-commerce events include:

- `currency` (ISO 4217 code, e.g. “SAR”, “USD”) – primary currency for values【1†L92-L99】【38†L125-L133】.
- `value` – total value (revenue) of the event/order (numeric).
- **Product Details:** `content_ids` (array of product IDs), `content_type` (e.g. “product”), `contents` (array of objects with `id` and `quantity`) for Meta; TikTok uses similar keys (`content_ids`, `content_type`); GA4 uses `items` array (each item: `item_id`, `item_name`, `price`, `quantity`, etc【43†L324-L332】【44†L567-L574】).
- `content_name`, `content_category` – optional names for product/content (Meta/TikTok).
- `num_items` or `quantity` – number of items.
- `search_string` – the search query (TikTok `Search` event).
- For checkout steps: `coupon` if a coupon applied, `shipping_tier`, `payment_type` (GA4 `add_payment_info` can include `payment_type` 【44†L526-L534】).
- `transaction_id` – unique order ID, required for purchases/refunds in GA4【44†L567-L574】【44†L627-L634】 and strongly recommended for Meta (used as `event_id`).
- **User Identifiers:** (hashed or raw as allowed) e.g. `em` (email), `ph` (phone), `fn` / `ln` (first/last name), `external_id`, `client_ip_address`, `client_user_agent` for Meta CAPI; TikTok supports similar matching fields. These help with ad attribution (especially hashed email/phone).
- `event_id` – a unique ID for deduplication (see section 4).

Example Payloads

- **Meta Pixel (browser)** – e.g. AddToCart:

```
fbq('track', 'AddToCart', {
  content_ids: ['SKU_123'],
  content_type: 'product',
  contents: [{id: 'SKU_123', quantity: 1}],
  currency: 'USD',
  value: 100.00
});
```

- **Meta Conversions API (server)** – e.g. Purchase:

```
{
  "data": [
    {
      "event_name": "Purchase",
      "event_time": 1670000000,
      "event_source_url": "https://example.com/checkout",
      "action_source": "website",
      "event_id": "ORDER_1058",
      "user_data": {
        "em": "<sha256(user@example.com)>",
        "ph": "<sha256(0551234567)>",
        "client_ip_address": "198.51.100.1",
        "client_user_agent": "Mozilla/5.0..."
      },
      "custom_data": {
        "currency": "USD",
        "value": 150.00,
        "content_ids": ["1058"],
        "content_type": "product",
        "contents": [{"id":"1058","quantity":1}]
      }
    }
  ]
}
```

- **TikTok Pixel (browser)** – e.g. ViewContent:

```
ttq.track('ViewContent', {
  content_type: 'product',
  content_id: 'SKU_123',
  content_name: 'T-Shirt',
  content_category: 'Clothing',
```

```
  currency: 'USD',
  value: 49.99
});
```

- **TikTok Events API (server)** – e.g. Purchase (JSON body):

```
{
  "pixel_code": "TIKTOK_PIXEL_ID",
  "event": "Purchase",
  "timestamp": 1670001234,
  "event_id": "ORDER_1058",
  "properties": {
    "currency": "USD",
    "value": 150.00,
    "content_ids": ["1058"],
    "content_type": "product",
    "content_name": "T-Shirt",
    "description": "Red T-Shirt"
  },
  "user": {
    "external_id": "<sha256(user@example.com)>"
  },
  "context": {
    "ip": "198.51.100.1",
    "user_agent": "Mozilla/5.0..."
  }
}
```

Notes: TikTok's Events API expects the pixel code, event name, timestamp (Unix), and an `event_id` for deduplication【21†L20-L27】. It also accepts user identifiers (e.g. hashed email/phone) under `user` or `context`.

- **Google (GTM + GA4)** – e.g. dataLayer push for purchase【44†L567-L574】:

```
window.dataLayer.push({
  event: 'purchase',
  transaction_id: 'ORDER_1058',
  currency: 'USD',
  value: 150.00,
  tax: 10.00,
  shipping: 5.00,
  coupon: 'SUMMER10',
  items: [
    {
      item_id: '1058',
      item_name: 'T-Shirt',
```

```
      item_brand: 'BrandX',
      item_category: 'Apparel',
      price: 140.00,
      quantity: 1
    }
  ]
});
```

Then configure GTM to send `purchase` event to GA4 and (optionally) Google Ads with these parameters. GA4 expects `transaction_id` and recommended fields as above【44†L567-L574】.

All parameters above should *match between browser and server* sends (same event_name, currency, value, content_ids, etc.) to ensure correct attribution【1†L82-L89】.

2.2. Required vs. Recommended

We summarize required fields per platform:

Parameter	Meta (Pixel/CAPI)	TikTok (Pixel/API)	GA4 / Google Ads (via GTM)
currency	*Strongly recommended* (required if `value` sent)【1†L92-L99】	Recommended for purchase events	*Required* at event level if `value` present【38†L125-L133】
value	For purchase/ checkout events (yes)	For purchase events (yes)	Yes (purchase value)【44†L567-L574】
content_ids / items	IDs of products (array) for ViewContent/ AddToCart/Purchase (recommended)	content_ids (array) for AddToCart/Purchase【20†L30-L33】【20†L48-L50】	`items` array (item_id, item_name, etc)【43†L324-L332】【44†L567-L574】
contents / items details	quantity, item info (optional object)	content_type, quantity, etc【20†L30-L33】	In `items`: item_name, item_category, price, quantity, etc【43†L324-L332】【44†L567-L574】
transaction_id	Recommended for Purchase (use as event_id)	Use as `event_id` if available	Required for purchase/ refund【44†L567-L574】【44†L627-L634】
event_id	Use same `event_id` for Pixel & CAPI to dedupe (see below)	Must provide identical `event_id` on Pixel and API for dedup【21†L20-L27】	Not used (GA4 dedupes by transaction_id)

Parameter	Meta (Pixel/CAPI)	TikTok (Pixel/API)	GA4 / Google Ads (via GTM)
user_data (email/phone etc)	Optional hashed user identifiers for matching	Optional hashed identifiers (TikTok calls it user or context data)	GA4 uses user_id if set (not needed for purchase monetization)

Sources: Google's guide specifies currency at event level and including all available e-commerce parameters【38†L125-L133】【43†L324-L332】. TikTok's docs list recommended parameters like `content_ids, currency, value` for AddToCart and Purchase【20†L30-L33】【20†L48-L50】. Meta's documentation emphasizes sending as much relevant data (currency, value, contents) as possible【1†L92-L99】.

3. Event Deduplication

Since we fire each event **twice** (browser Pixel + server API), we must deduplicate to avoid double-counting. **Meta and TikTok both require a unique** `event_id` **parameter** on both browser and server. Use the same string for both sends (e.g. `"ORDER_1234"` or any unique ID). The platforms will keep the first and drop duplicates【21†L20-L27】.

- **Meta:** In CAPI payload, set `"event_id": "<ORDER_ID>"`. In the Pixel call (e.g. via `fbq('track', ..., {event_id: ...})` or use a data attribute), use the same `event_id`. Meta will then dedupe. Although not documented in our sources, this is standard practice in the Meta developer guides (the PixelFlow doc explicitly notes matching names between Pixel and CAPI【1†L82-L89】).
- **TikTok:** Must include the same `event_id` parameter in both Pixel and Events API calls【21†L20-L27】. TikTok dedupes if events overlap (same event name + event_id within 48h)【21†L39-L47】.

For example, if order #1058 is placed, you might generate:

```
$event_id = 'order_1058';
```

Then in browser:

```
fbq('track', 'Purchase', { /*...,*/ event_id: event_id });
ttq.track('Purchase', { /*...,*/ event_id: event_id });
```

And in server API calls, include `"event_id": "order_1058"`. This ensures each purchase counts once.

> **Deduplication note:** If you only send an event on one side (browser *or* server), event_id isn't needed. But we will use both paths for robustness, so **event_id is mandatory**【21†L20-L27】.

4. User Data & Privacy

User Identifiers (for matching): To maximize matching, include non-sensitive identifiers hashed (SHA256 hex) in server calls: email (`em`), phone (`ph`), name (`fn` , `ln`), external customer ID, IP, user agent. Meta's CAPI documentation and TikTok's recommend passing available IDs. *Do not send any prohibited or sensitive personal info*. For Meta, see their Advanced Matching guidelines (e.g. `em` , `ph` hashed) and for TikTok, similar matching keys. These must be SHA256 hashed **lowercase hex** before sending.

Cookie Consent: Ensure you respect user consent. Only fire the Pixel/Events if allowed. Provide a "tracking on/off" switch in Marketing Settings, and integrate with your cookie banner. Many platforms (Meta/TikTok) will require user consent under GDPR. (TikTok notes: "avoid sending sensitive data"【20†L17-L21】, and always comply with privacy policies.)

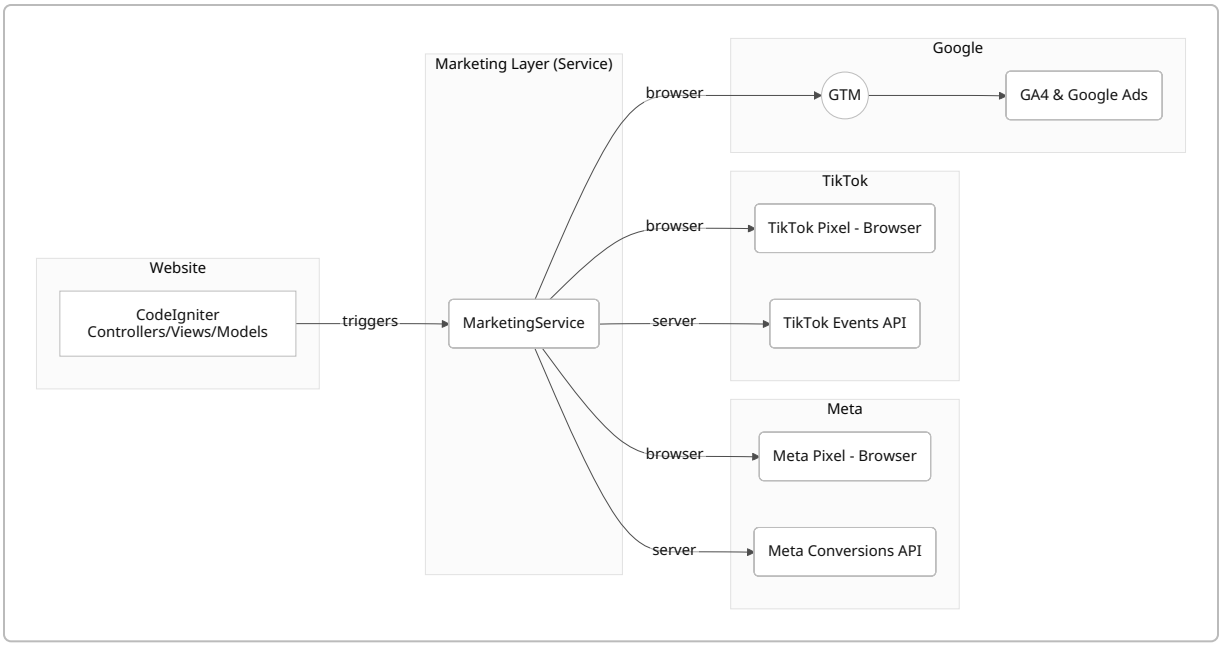
Hashing Example (PHP):

```
function sha256($input) {
    return strtolower(hash('sha256', $input));
}
$userData = [
    'em' => sha256($userEmail),
    'ph' => sha256($userPhone),
    // etc.
];
```

Ensure SSL is enabled and tokens kept secret (CAPI tokens on server only).

5. CodeIgniter 4 Architecture

We will add new controllers, models, and services. Here's an overview diagram:



Key components:

- **Routes:** Define new routes (in `app/Config/Routes.php`) for Marketing admin pages (e.g. `/admin/marketing/settings`, `/admin/marketing/logs`), and an API route for server event calls if needed (though most calls are internal).
- **Controllers:**
- `MarketingController` – handles Admin UI (settings form, logs page).
- Optionally `Api\MarketingWebhook` for any webhook endpoints (not required here).
- **Models:**
- `MarketingSettingModel` – stores pixel IDs, tokens, toggle, test mode, etc.
- `EventLogModel` – logs all fired events (fields: `id`, `platform`, `event_name`, `payload`, `response`, `status`, `created_at`).
- Extend your existing `Order` model/table to include `marketing_event_id` or status if needed.
- **Services / Libraries:**
- `MetaService` – methods to send events to Meta (pixel insertion, CAPI calls).
- `TikTokService` – methods to send Pixel/Events API.
- `GoogleService` – pushes to `dataLayer` (browser only, via GTM).
- `MarketingService` (high-level) – central point to call Pixel and server methods on each event.

For example, in CI4 you might have `app/Services/MarketingService.php` with:

```
public function trackEvent(string $eventName, array $data) {
    // Send browser pixel (maybe by injecting JS) and queue server calls:
    MetaService::sendPixelEvent($eventName, $data);
    MetaService::sendCAPIEvent($eventName, $data);
    TikTokService::sendPixelEvent($eventName, $data);
    TikTokService::sendEventsAPI($eventName, $data);
    GoogleService::pushDataLayer($eventName, $data);
```

```
    // Log event
    EventLogModel::log($eventName, $data, 'queued');
}
```

Each service uses the IDs/tokens from the settings (loaded from DB), and handles HTTP requests or JS snippet rendering.

Database Schema (example):

- `marketing_settings` table: `{ id, meta_pixel_id, meta_capi_token, tiktok_pixel_id, tiktok_api_token, gtm_id, ga4_id, google_ads_id, enabled, test_mode, updated_at }`.
- `event_logs` table: `{ id, platform, event_name, payload TEXT, response TEXT, status ENUM('sent','error'), created_at }`.
- Optionally, in `orders` table add `{ marketing_event_id VARCHAR, status ENUM('placed','delivered','cancelled','refunded'), payment_method ENUM('COD','Online') }` to track events and COD statuses.

6. Admin Marketing Panel

Under **Admin → Marketing → Tracking Settings**, present fields:

- **Toggle Enable Tracking:** On/Off checkbox. (If Off, skip all tracking calls.)
- **Meta Pixel ID:** (text) validated as numeric ID.
- **Meta Conversions API Token:** (text) (store encrypted).
- **TikTok Pixel ID:** (text).
- **TikTok Events API Token:** (text) (encrypted).
- **GTM Container ID:** (text) format `GTM-XXXX`.
- **GA4 Measurement ID:** (text) format `G-XXXX`.
- **Google Ads Conversion ID/Label:** (text, optional).
- **Test Mode:** Checkbox. If on, all events go to debug/logging (and not counted in production).

Use server-side validation (e.g. required fields, correct formats). **Secure storage:** Save tokens encrypted (use CI4's `Encryption` library or store in `.env` via the Model). Display masked or not at all when showing existing settings.

Also add **Marketing → Event Logs** page: list recent events (columns: Time, Platform, Event, Data, Status). Allow filtering by status (error/success). This helps QA/debug.

7. Frontend (Browser) Integration

7.1. GTM + dataLayer

Add the GTM snippet to every page (usually in `<head>`):

```
<script>
window.dataLayer = window.dataLayer || [];
</script>
<script async src="https://www.googletagmanager.com/gtm.js?id=<?=$gtm_id?>"></
script>
```

(with appropriate no-script fallback). This should use the ID from settings. Then for each event, push to `dataLayer`. For example, on AddToCart button click:

```
dataLayer.push({
  event: 'add_to_cart',
  items: [
    { item_id: '123', item_name: 'Popcorn Maker', item_category: 'Kitchen',
price: 100.00, quantity: 1 }
  ],
  currency: 'SAR',
  value: 100.00
});
```

Configure GTM triggers for each event name (`add_to_cart`, `purchase`, etc) and tags to send to GA4 & Google Ads. GA4 tags will use GA4 Measurement ID; Google Ads tag uses conversion ID/label.

Reference: Google's ecommerce guide shows this pattern (dataLayer with `event`, `currency`, `items` etc)【43†L324-L332】【44†L567-L574】.

7.2. Meta Pixel

Include the Meta Pixel base code once (in a global footer or via view composer):

```
<!-- Meta Pixel Code -->
<script>
!function(f,b,e,v,n,t,s)
{if(f.fbq)return;n=f.fbq=function(){n.callMethod?
n.callMethod.apply(n,arguments):n.queue.push(arguments)};
 if(!f._fbq)f._fbq=n; n.push=n; n.loaded=!0; n.version='2.0';
 n.queue=[]; t=b.createElement(e); t.async=!0;
 t.src=v; s=b.getElementsByTagName(e)[0];
 s.parentNode.insertBefore(t,s);}
(window, document,'script','https://connect.facebook.net/en_US/fbevents.js');
fbq('init', 'META_PIXEL_ID' /* Insert actual ID */);
fbq('track', 'PageView');
</script>
<noscript><img height="1" width="1" style="display:none"
src="https://www.facebook.com/tr?id=META_PIXEL_ID&ev=PageView&noscript=1"
```

```
/></noscript>
<!-- End Meta Pixel Code -->
```

Replace `META_PIXEL_ID` with the ID from settings. To enable Advanced Matching, you can pass user info to `fbq('init', ...)`:

```
fbq('init', 'PIXEL_ID', {
  em: sha256(user.email),
  ph: sha256(user.phone)
});
```

and always call `fbq('track', eventName, parameters)`. For example:

```
fbq('track', 'ViewContent', {
  content_ids: ['123'], content_type: 'product', currency: 'SAR', value: 100.00
});
```

7.3. TikTok Pixel

Include TikTok's base pixel script (place in `<head>`):

```
<!-- TikTok Pixel Code -->
<script>
!function (w, d, t) {
  w.TiktokAnalyticsObject = t; var ttq = w[t] = w[t] || [];
  ttq.methods =
["page","track","identify","instances","debug","on","off","once","ready","alias","group","enableC
  ttq.setAndDefer = function(t, e) { t[e] = function() {
t.push([e].concat(Array.prototype.slice.call(arguments,0))) }};
  for(var i=0; i<ttq.methods.length; i++) { ttq.setAndDefer(ttq,
ttq.methods[i]); }
  ttq.instance = function(t) { var e = ttq._i[t] || []; var n = ttq;
n.methods.forEach(function(t) { n.setAndDefer(e, t) }); return e; };
  ttq.load = function(id) { ttq._i = ttq._i || {}; ttq._i[id] = [];
ttq._i[id]._u = 'https://analytics.tiktok.com/i18n/pixel/events.js';
    var n = 'ttq'; var o = document.createElement("script"); o.type = "text/
javascript"; o.async = !0; o.src = ttq._i[id]._u;
    var a = document.getElementsByTagName("script")[0];
a.parentNode.insertBefore(o, a); };
  ttq.load('TIKTOK_PIXEL_ID');
  ttq.page();
}(window, document, 'ttq');
```

```
</script>
<!-- End TikTok Pixel Code -->
```

Replace `TIKTOK_PIXEL_ID` with your ID. Then track events via:

```
ttq.track('ViewContent', {
  content_type: 'product',
  content_id: '123',
  currency: 'SAR',
  value: 49.99
});
```

As with Meta, you can pass user data for advanced matching:

```
ttq.identify('user_email_hash', {
  email: sha256(userEmail),
  phone_number: sha256(userPhone)
});
```

(TikTok's advanced matching docs recommend setting `identify` *or including context user fields.)*

8. Server-side (Conversions/Event API) Endpoints

Implement server calls in your services. For each event (AddToCart, Purchase, etc.), call the respective platform API **after** the order action occurs (e.g. on order completion hook or immediately after `Order` model save).

- **Meta Conversions API:**
  Endpoint: `https://graph.facebook.com/v16.0/{pixel_id}/events?access_token={CAPI_token}`.
  Use POST with JSON body as shown above. Handle HTTP response (200 success or errors). Retry on failure (respect rate limits ~<50 QPS per pixel). Log the request and response in `event_logs`.
  Example PHP using cURL/Guzzle:

```
$payload = [ /* as in example above */ ];
$res = $httpClient->post(
    "https://graph.facebook.com/v16.0/{$pixel_id}/events?
access_token={$capiToken}",
    ['json' => $payload]
);
$result = $res->getBody();
```

```
EventLogModel::log('MetaCAPI', 'Purchase', json_encode($payload), $result,
'sent');
```

- **TikTok Events API:**
Endpoint: e.g. `https://business-api.tiktok.com/open_api/v1.3/pixel/trigger/`. (Check the latest TikTok API docs; older versions used v1.2 or v1.3.) Pass your App ID and Access Token (if required) in headers or query params.
The JSON body format is like the example above.
Example PHP:

```
$payload = [ /* as TikTok JSON above */ ];
$headers = [
  'Access-Token: ' . $tiktokToken,
  'Content-Type: application/json'
];
$ch = curl_init("https://business-api.tiktok.com/open_api/v1.3/pixel/trigger/?
app_id={$tiktokAppId}");
curl_setopt($ch, CURLOPT_POST, true);
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($payload));
curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
$response = curl_exec($ch);
EventLogModel::log('TikTokAPI', 'Purchase', json_encode($payload), $response,
'sent');
```

Note: Include `pixel_code` in payload. Ensure to set `timestamp` (TikTok expects seconds). Implement retries/backoff on 500 errors.

- **Google (Server-Side):** Not required for basic e-commerce tracking. (If using Google Ads server-side API or GA4 Measurement Protocol, that's advanced and out of scope.) We rely on GTM on frontend.

Error Handling & Rate Limits: Both Meta and TikTok have API limits. Check responses for rate-limit HTTP 4xx/5xx codes. Log any errors in `event_logs` with status "error" and the message. In CI4, you can implement try/catch around the API calls and record failures. Use exponential backoff for retries (or simply log and alert, so developer can review later).

9. COD Orders & Status Updates

For **Cash-on-Delivery (COD)** orders, special care is needed. On order placement (COD payment), immediately fire the `Purchase` event with the order value (since the customer *placed* the order). But this does not guarantee revenue: the customer might refuse later. To measure real ROAS, you can send follow-up events:

- If a COD order is **delivered** (finalized), send a server `Purchase` event or update (some platforms allow specifying revenue later, but simplest is to re-send Purchase with same event_id or send a custom "Order Completed" event).

- If a COD order is **cancelled/refunded**, send a `Refund` event via GA4 (GA4: `refund` event with `transaction_id`)【44†L627-L634】. For Meta/TikTok, you might send a lower-value purchase or a custom event to negate. (Meta CAPI supports a “Purchase” with zero value, or you can use “Offline Conversion” APIs but that’s beyond scope.)

Implementation: In your Order model/controller, after saving an order: 1. On placement (COD or online), call `MarketingService->trackEvent('Purchase', $data)`. 2. When an order status later changes (e.g. to "Delivered" or "Cancelled"), trigger another call:

- For **Delivered**, you could re-send a `Purchase` or custom event “Purchase_Completed” with same event_id but maybe now with `value=0` if it was refunded, or just trust the first one.
- For **Cancelled**, send a `refund` or adjust events. At minimum, log internally and exclude this order from ROI calculations.

Important: Record `payment_method` and `status` in your orders table. Use this to decide when to trigger events. For example:

```
if ($order->payment_method === 'COD') {
    MarketingService::trackEvent('Purchase', $orderData);
}
```

Later:

```
if ($order->status === 'Cancelled') {
    GoogleService::pushDataLayer('refund', [...]); // GA4 refund
    // For Meta/TikTok, could send a 'Purchase' with negative value or skip.
}
```

This ensures your analytics align with actual completed sales.

10. Test Mode and Debugging

- **Test Mode:** In admin panel, a “Test Mode” switch should toggle verbose logging and use of Meta/TikTok’s test endpoints (if available). In test mode, do not affect production reporting. E.g. for Meta CAPI, you can append `&test_event_code=TEST123` to the API call to mark it as a test event (Facebook provides a test event code in Events Manager). For TikTok, use any test debug parameter if provided. At minimum, log everything in detail.

- **Event Log UI:** The admin UI **Event Logs** should show each event attempt, its payload (JSON), and platform response. Show timestamp, event name, platform, and status. Highlight errors in red. This is critical for QA.

- **Verify Events:**

- **Meta:** Use **Meta Events Manager Test tool** to verify events in real-time【1†L99-L107】. (The PixelFlow doc notes: use Events Manager to confirm required params【1†L99-L107】.) Check that `Purchase` has value/currency and right content_ids.
- **TikTok:** Use TikTok's Event Manager or Pixel Helper Chrome extension to verify Pixel events and ensure no missing params.
- **Google:** Use GA4 DebugView and GTM Preview to ensure events fire with correct parameters.

11. Security & Privacy

- **Secure tokens:** Do NOT expose API tokens in JavaScript or client-side. All server calls should fetch tokens from secure storage on server.
- **Data Protection:** Hash or omit PII unless absolutely necessary. Ensure TLS (HTTPS) is used for all API calls.
- **Rate Limits:** Meta CAPI has a limit (generally 50 calls per second per pixel). Implement basic retry logic. TikTok also has limits; if you hit a limit, slow down calls or batch them. (Batching can be done by sending multiple `data` objects in one CAPI request.)

12. Implementation Checklist

1. **Database Setup:** Create tables/migrations:
2. `marketing_settings` (for IDs, tokens, flags).
3. `event_logs` (for debugging).
4. Alter `orders` if needed (add `marketing_event_id`, `payment_method`, `status`).
5. Update CI4 Entities/Models accordingly.
6. **Admin UI:** Build Marketing Settings form and Event Log page:
7. Fields listed in section 6, with validation.
8. Encrypt tokens when saving.
9. Load settings into config/service at runtime.
10. **Include Pixel/Tag Snippets:**
11. Insert Meta Pixel and TikTok Pixel scripts (use settings IDs).
12. Insert GTM snippet with container ID.
13. Ensure `window.dataLayer` exists.
14. **DataLayer Implementation:**

15. Write JS in views or templates to push events: page load (implicitly page_view by GTM), product views, add to cart, etc. Use global `dataLayer.push({event: ..., items: [...], currency: ..., value: ...})`.

16. Attach these pushes to appropriate actions (e.g. on button click or on page load for ViewContent).

17. **MarketingService:** Develop a service class:

18. `trackEvent($name, $data)` method that: logs event locally, calls Pixel JS (if needed), and queues server API calls.

19. Inside, call specialized methods: `MetaService::track($...)`, `TikTokService::track($...)`, `GoogleService::track($...)`.

20. Generate `event_id` (e.g. `'order_'.$orderId`).

21. **Server API Calls:**

22. **MetaService:** Implement `sendCAPIEvent($eventName, $params)` to POST to Graph API. Include `event_id`, `user_data`, `custom_data`.

23. **TikTokService:** Implement `sendEventsAPI($eventName, $params)` to POST to TikTok endpoint. Map our `$eventName` to TikTok's (names are usually identical for standard events). Include `pixel_code`, `timestamp`, `event_id`, `properties`, `user`, `context`.

24. **Error Handling:** On API call, log success or failure in `event_logs`. If error, retry once or log for review.

25. **E-commerce Integration:**

26. On product detail page load: call `trackEvent('ViewContent', {...})`.

27. On search: `trackEvent('Search', {search_string: '...'}).`
28. On AddToCart action: call `trackEvent('AddToCart', { /*items, currency, value*/})`.
29. On cart view: `trackEvent('ViewCart', {...})`.
30. On checkout start: `trackEvent('InitiateCheckout', {...})`.
31. On add payment info: `trackEvent('AddPaymentInfo', {...})`.
32. On order completion (online or COD): `trackEvent('Purchase', {...transaction_id, currency, value, items...})`. Include `transaction_id` = order ID.

33. On registration: `trackEvent('CompleteRegistration', {})`. On contact form: `trackEvent('Lead', {})`.

34. **COD Handling:**

35. If payment is COD, you still send `Purchase` at placement (value = order total). Record `payment_method='COD'`.

36. On status change to Cancelled/Failed: consider sending a GA4 `refund` event (via `dataLayer.push({event:'refund', transaction_id:..., value:...})`). You may also notify Meta/TikTok by sending zero-value purchase or skip (these platforms don't have a direct refund API in CAPI).

37. On Delivered: optionally update event or rely on initial Purchase event.

38. **Testing & QA:**

39. Enable **Test Mode** in settings. Perform actions on the site (use test product, test transaction). Check the **Event Logs** page for entries.

40. In Meta Events Manager (Test Events), verify that your test events appear with correct params 【1†L99-L107】.
41. In TikTok Events Manager or with the Pixel Helper, verify events.

42. In GA4 DebugView, see `purchase`, `add_to_cart`, etc., with correct values.

43. **Security Review:**

    - Confirm tokens are not leaked to frontend (inspect source code).
    - Verify consent mode if needed (only fire tags after opt-in).

44. **Deployment:**

    - Move `marketing_settings` tokens to production keys.
    - Turn off Test Mode.
    - Monitor logs/errors for a day to ensure events are flowing.

By following this plan, you will add a robust, centralized tracking layer to your CodeIgniter 4 site. The system will reliably send all user actions (pages, clicks, transactions) to Meta, TikTok, and Google with consistent data, enabling accurate campaign measurement and optimization.
